Cómo llega HTTP al código servidor — versión ampliada

Para seguir la explicación en clase, empieza por la versión corta. Aquí se sigue cada petición a través de los procesos y se comparan tres despliegues concretos; la demo de Docker es formativa y no añade una nota independiente.

Un servidor web sabe recibir HTTP, pero no ejecuta por sí mismo cualquier lenguaje. Necesita un mecanismo que conecte la petición con el runtime y con el código de la aplicación. Las tecnologías han evolucionado para evitar crear un proceso costoso en cada petición y para gestionar concurrencia, aislamiento y despliegue.

Modelo general

```
cliente -> servidor web o proxy -> runtime/contenedor -> aplicación -> datos o servicios
```

Las piezas pueden vivir en el mismo proceso, en procesos separados o incluso en máquinas distintas. La arquitectura monolítica no obliga a colocar la base de datos dentro de la aplicación.

Mecanismos principales

Mecanismo	Cómo funciona	Ventaja	Coste o riesgo
CGI	Ejecuta un proceso por petición	Modelo simple y aislado	Crear procesos limita el rendimiento
FastCGI	Mantiene procesos preparados y reutilizables	Evita arrancar el runtime en cada petición	Requiere gestionar procesos y comunicación
Módulo del servidor	El runtime se carga dentro del servidor web	Comunicación directa	Menor aislamiento entre servidor y aplicación
Proceso persistente	La aplicación escucha peticiones continuamente	Buen control de concurrencia y estado interno	Hay que gestionar ciclo de vida y recursos
Contenedor de aplicaciones	Un runtime aloja componentes bajo un contrato común	Despliegue, configuración y servicios compartidos	Añade convenciones y complejidad

PHP suele desplegarse detrás de un servidor web mediante PHP-FPM/FastCGI. Una aplicación Spring Boot suele mantener un proceso Java persistente con un servidor HTTP embebido. Ninguno de esos modelos es universalmente mejor: responden a ecosistemas y necesidades diferentes.

Para observar una combinación distinta, la demo Nginx + fcgiwrap + Python CGI permite seguir una petición de navegador a servidor web, de ahí por FastCGI a `fcgiwrap` y finalmente a un **script CGI nuevo por petición**. La tarea breve pide modificar una respuesta y explicar el recorrido. Que haya un tramo FastCGI no implica que el script Python permanezca vivo entre peticiones, como sí lo hace un trabajador PHP-FPM.

Recorrido 1: CGI clásico

CGI es un **contrato** para pasar datos de una petición a un programa y recoger su salida. En el modelo clásico, el servidor prepara información como el método y los parámetros, ejecuta un proceso para esa petición y usa la salida del programa para construir la respuesta:

```
cliente --HTTP--> servidor web --datos CGI--> nuevo proceso del script
cliente <--HTTP-- servidor web <--cabeceras + cuerpo-- script finalizado
```

Una variable de entorno como `QUERY_STRING` puede transportar los parámetros de la URL; el programa imprime primero campos como `Content-Type`, después una línea en blanco y finalmente el contenido. La ventaja es que el modelo resulta fácil de observar y cada ejecución se separa de las anteriores. El coste de crear un proceso por petición puede ser importante cuando llegan

muchas peticiones. **Nginx no ejecuta por sí mismo un script CGI arbitrario**: la siguiente demo necesita un intermediario.

Recorrido 2: Nginx, fcgiwrap y Python CGI

En la demo Docker de UD01, el navegador accede a `/` y a `/cgi-bin/saludo.py`:

1. Nginx recibe las dos peticiones HTTP. Para `/`, sirve `web/index.html`: no interviene Python.
2. Para `/cgi-bin/saludo.py?nombre=Ada`, Nginx usa `fastcgi_pass` y comunica la petición al proceso **fcgiwrap** mediante FastCGI. Le facilita la ruta del script (`SCRIPT_FILENAME`) y la consulta (`QUERY_STRING`).
3. **fcgiwrap permanece en marcha**, pero lanza `saludo.py` como un **proceso CGI nuevo para esa petición**. Python lee el nombre, genera HTML y escribe `Content-Type` antes del cuerpo.
4. fcgiwrap devuelve la salida por FastCGI a Nginx; Nginx envía la respuesta HTTP al navegador. En la siguiente petición, fcgiwrap volverá a lanzar el script.

```
navegador --HTTP--> Nginx --FastCGI--> fcgiwrap (persistente)
                                          | CGI
                                          v
                                   script Python (por petición)
```

FastCGI describe el enlace Nginx–fcgiwrap, no la duración del script Python. El PID mostrado por la demo ayuda a investigar qué proceso ejecutó el script, pero un PID puede reutilizarse: no es una prueba aislada de que se haya conservado el mismo proceso. Editar `saludo.py` en el equipo modifica el archivo montado en el contenedor; al recargar se ejecuta la versión nueva sin reconstruir la imagen.

La tarea formativa permite cambiar la respuesta y dibujar el recorrido. No hace falta estudiar Python ni Docker como objetivos de esta unidad: se observa **quién recibe HTTP y quién ejecuta el código**.

Recorrido 3: Nginx y PHP-FPM

En una instalación habitual de PHP, Nginx comunica las peticiones a PHP-FPM por FastCGI. PHP-FPM mantiene **trabajadores preparados** que atienden peticiones sucesivas; el runtime PHP ya está disponible dentro de ellos:

```
navegador --HTTP--> Nginx --FastCGI--> trabajador PHP-FPM
                                          (continúa tras la respuesta)
```

El código PHP se evalúa para cada petición, pero **no se arranca un intérprete nuevo como proceso para cada una** en el modelo habitual PHP-FPM. Esa es la diferencia importante con la combinación `fcgiwrap` + script CGI. Ambos usan un tramo FastCGI, pero el ciclo de vida del código de aplicación no es igual. No hay que montar PHP-FPM para resolver la tarea Python: aquí sirve de comparación conceptual y se trabajará más adelante en el módulo.

Recorrido 4: Spring Boot con Tomcat embebido

Hello Server arranca una aplicación Java que permanece activa junto con Tomcat embebido:

```
navegador --HTTP--> proceso Java: Tomcat --> controlador --> HTML/JSON
             <-- respuesta HTTP por el mismo servidor activo --
```

Tomcat recibe HTTP, Spring dirige la petición al controlador y el código Java prepara la respuesta. Visitar `/api/hello` dos veces **no inicia dos JVM**: la misma aplicación atiende múltiples peticiones durante su vida. Eso no significa que todo se ejecute secuencialmente; el servidor gestiona concurrencia y recursos compartidos. Tampoco convierte una respuesta JSON mínima en una API REST completa: ese diseño se aborda en UD02.

Caso	Proceso que sigue activo	¿Se crea un proceso del script por petición?
CGI clásico	Servidor web	Sí, para el programa CGI
Nginx + fcgiwrap + Python	Nginx y fcgiwrap	Sí, para `saludo.py`

Caso	Proceso que sigue activo	¿Se crea un proceso del script por petición?
Nginx + PHP-FPM	Nginx y trabajadores PHP-FPM	No en el despliegue habitual
Hello Server	Aplicación Java con Tomcat	No: la JVM ya está en marcha

Estas comparaciones describen **mecanismos de ejecución**, no una escala universal de tecnologías «mejores» y «peores». También puede haber límites de trabajadores, conexiones y memoria en cualquiera de los modelos.

Concurrencia y estado

Un proceso persistente atiende muchas peticiones durante su vida. Esto mejora el rendimiento, pero obliga a pensar en:

- datos compartidos entre hilos o tareas;
- conexiones y pools limitados;
- fugas de memoria;
- bloqueos y operaciones lentas;
- cierre ordenado y recuperación ante fallos.

El estado de un usuario no debe guardarse en variables globales improvisadas. Las sesiones, bases de datos, cachés y tokens tienen contratos específicos que se estudiarán más adelante.

Por ejemplo, si dos navegadores piden `/api/hello` casi al mismo tiempo, el servidor debe atenderlos sin mezclar la respuesta de un usuario con la del otro. Que el proceso Java sea persistente **no** autoriza a guardar datos de cada usuario en un campo compartido del controlador. Más adelante veremos persistencia y sesiones con contratos explícitos; aquí basta reconocer el riesgo y situar en qué proceso vive el código.

Mecanismo frente a arquitectura

No se deben confundir estas decisiones:

- **Mecanismo de ejecución**: cómo se invoca el código ante una petición.
- **Arquitectura de aplicación**: cómo se organizan responsabilidades dentro del código.
- **Topología de despliegue**: en qué procesos y máquinas se ejecutan los componentes.

Una aplicación monolítica puede ejecutarse como proceso persistente detrás de un proxy y consultar una base de datos remota. Son decisiones independientes.

Pregunta	En Hello Server detrás de Nginx
¿Qué mecanismo invoca nuestro código?	Tomcat/Spring dentro de un proceso Java persistente
¿Cómo está organizada la aplicación?	Un proyecto con controlador; puede seguir siendo monolítico
¿Dónde se ejecuta?	Java en su proceso, Nginx delante como proxy, datos donde se haya desplegado la persistencia

Añadir un proxy no convierte automáticamente una aplicación monolítica en microservicios, del mismo modo que usar FastCGI no convierte un script CGI en un trabajador de aplicación persistente.

Comprobación

Compara PHP-FPM y una aplicación Spring Boot ejecutable:

1. Qué proceso recibe inicialmente HTTP.
2. Qué proceso ejecuta el código de negocio.
3. Qué componentes permanecen vivos entre peticiones.

4. Qué recurso agotado podría impedir atender nuevas peticiones.

Después, usando los dos READMEs de la demo Python y Hello Server, dibuja `GET /cgi-bin/saludo.py` y `GET /api/hello`. Señala con un círculo los componentes que continúan vivos tras responder y subraya el componente que genera el HTML o JSON en cada caso. No deduzcas el mecanismo solo a partir de una cabecera `Server`: puede corresponder al primer intermediario.

Esta evidencia trabaja RA1.c.